Singly Linked Lists in Python

A Complete Beginner-to-Builder Chapter

Learning Promise

By the end of this chapter, you should be able to write every singly linked list operation from scratch without memorizing pointer changes. You will know why, when, where, and how every reference changes.

Contents

1 Singly Linked Lists in Python

1.1 The Big Problem Before Linked Lists

Problem

Python lists are excellent general-purpose containers, but inserting or deleting near the front requires shifting many elements. If a list contains one million items and we insert at index 0, many references must move one position to the right.

Strict Teaching Flow for the Chapter

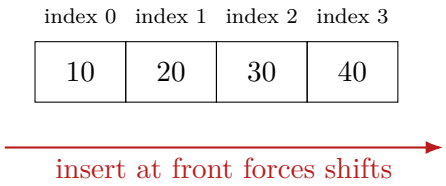
Every major operation follows the same learning order:

1. Problem
2. Why current solution is insufficient
3. Need
4. Intuition
5. Visualization
6. Logic
7. Dry Run
8. Pseudo Code
9. Python Code
10. Code Output
11. Memory Diagram
12. Time Complexity
13. Common Mistakes
14. Edge Cases
15. Interview Perspective
16. Practice Questions

For every method implementation, the chapter includes complete code, sample driver code, console output, output explanation, memory before and after, pointer movement, a dry run table, edge cases, complexity, interview discussion, and a mini exercise.

1.2 Why the Current Solution Is Insufficient

A Python list stores references in a contiguous dynamic array. This gives fast index access, but middle insertions and deletions are expensive because positions after the change must shift.



1.3 The Need

We need a structure where elements do not need to be physically side by side. Instead of moving many items, we want to rewire references.

Intuition

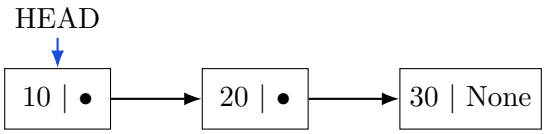
A singly linked list is a chain of independent node objects. Each node stores data and a reference to the next node. The list object stores only the first reference, called `head`.

1.4 Core Intuition: Nodes and References

A node is a small object with two fields:

$$\text{Node} = \boxed{\text{data} \mid \text{next}}$$

The `next` field does not copy the next node. It stores a reference to the next node object. The last node stores `None`.



1.5 Complete Reference Implementation Grows Gradually

We will start with `Node`, then `LinkedList`, then add one method at a time. To keep learning continuous, examples reuse the evolving list named `ll`. Fresh lists are introduced only for empty-list or single-node edge cases.

1.6 The Full Class Code After All Methods

Keep this as the final reference. Each method is explained separately afterward.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.n = 0

    def __len__(self):
        return self.n

    def is_empty(self):
        return self.head is None

    def display(self):
        current = self.head
        while current is not None:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        self.n += 1

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            self.n += 1
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node
        self.n += 1

    def insert_after(self, target, data):
        current = self.head
        while current is not None:
            if current.data == target:
                new_node = Node(data)
                new_node.next = current.next
                current.next = new_node
                self.n += 1
                return True
            current = current.next
        return False

    def insert_before(self, target, data):
        if self.head is None:
            return False
        if self.head.data == target:
            self.insert_at_beginning(data)
            return True
        previous = self.head
```

```
        current = self.head.next
        while current is not None:
            if current.data == target:
                new_node = Node(data)
                previous.next = new_node
                new_node.next = current
                self.n += 1
                return True
            previous = current
            current = current.next
        return False

    def insert_at_position(self, position, data):
        if position < 0 or position > self.n:
            raise IndexError("position out of range")
        if position == 0:
            self.insert_at_beginning(data)
            return
        previous = self.head
        for _ in range(position - 1):
            previous = previous.next
        new_node = Node(data)
        new_node.next = previous.next
        previous.next = new_node
        self.n += 1

    def search(self, value):
        current = self.head
        position = 0
        while current is not None:
            if current.data == value:
                return position
            current = current.next
            position += 1
        return -1

    def update(self, old_value, new_value):
        current = self.head
        while current is not None:
            if current.data == old_value:
                current.data = new_value
                return True
            current = current.next
        return False

    def delete_first(self):
        if self.head is None:
            return None
        deleted_value = self.head.data
        self.head = self.head.next
        self.n -= 1
        return deleted_value

    def delete_last(self):
        if self.head is None:
            return None
```

```
        if self.head.next is None:
            deleted_value = self.head.data
            self.head = None
            self.n -= 1
            return deleted_value
        previous = self.head
        current = self.head.next
        while current.next is not None:
            previous = current
            current = current.next
        deleted_value = current.data
        previous.next = None
        self.n -= 1
        return deleted_value

    def delete_by_value(self, value):
        if self.head is None:
            return False
        if self.head.data == value:
            self.delete_first()
            return True
        previous = self.head
        current = self.head.next
        while current is not None:
            if current.data == value:
                previous.next = current.next
                self.n -= 1
                return True
            previous = current
            current = current.next
        return False

    def delete_by_position(self, position):
        if position < 0 or position >= self.n:
            raise IndexError("position out of range")
        if position == 0:
            return self.delete_first()
        previous = self.head
        for _ in range(position - 1):
            previous = previous.next
        deleted_node = previous.next
        previous.next = deleted_node.next
        self.n -= 1
        return deleted_node.data

    def reverse(self):
        previous = None
        current = self.head
        while current is not None:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node
        self.head = previous

    def clear(self):
```

```
        self.head = None
        self.n = 0

    def to_list(self):
        result = []
        current = self.head
        while current is not None:
            result.append(current.data)
            current = current.next
        return result

    @classmethod
    def from_list(cls, values):
        linked_list = cls()
        for value in values:
            linked_list.insert_at_end(value)
        return linked_list

    def __iter__(self):
        current = self.head
        while current is not None:
            yield current.data
            current = current.next

    def __str__(self):
        return " -> ".join(str(value) for value in self) + " -> None"
```

1.7 Method-by-Method Mastery

Node

Method focus: Node. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** We need a container for one value plus a way to find the next value.
2. **Why we need this method.** Without nodes, there is no chain; a linked list is built from node objects.
3. **Visual explanation.** Think of one node as a two-compartment box: the left side stores the value, and the right side stores the address of the next box. A brand-new `Node(10)` is not attached to any list yet, so its right side is `None`.
4. **Algorithm thinking.** A node must be safe even before it is connected. Therefore, construction should always finish with a valid value field and a valid next field. The safest default for `next` is `None`because there is no known successor at creation time.
5. **Pseudo code.**

```
receive data
create a new node object
store data inside the node
set node.next to None because it has no successor yet
return the initialized node object
```

6. **Python implementation.**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

7. **Example program.**

```
node = Node(10)
print(node.data)
print(node.next)
```

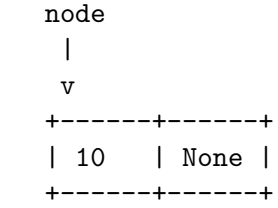
8. **Console Output.**

```
10
None
```

9. **Explain output line-by-line.** First line prints the stored value. Second line prints `None`because the node has no successor yet.
10. **Memory Diagram Before.** No node object exists.

    `node variable: undefined`

11. **Pointer Changes.** The variable `node` begins referring to the new object; `node.next` refers to `None`.
12. **Memory Diagram After.**



13. Dry Run Table.

Step	Action	State
1	call `Node(10)`	new object allocated
2	set data	data is 10
3	set next	next is None

14. Complexity. Time $O(1)$, space $O(1)$ per node.

15. Common mistakes. Forgetting `self.next = None`; using `next` as a local variable only.

16. Edge cases. Data can be any Python object: integer, string, list, or custom object.

17. Interview discussion. A node is the atomic unit; explain that `next` stores a reference, not a copy.

18. Mini Exercise. Create nodes with values 1, 2, 3 and manually connect them.

LinkedList and `__init__`

Method focus: LinkedList and `__init__`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** We need one object to remember where the chain starts.
2. **Why we need this method.** If we lose the first node reference, the entire chain becomes unreachable.
3. **Visual explanation.** The linked-list object is the control panel, not a node. In an empty list, the control panel's `head` arrow points nowhere, so the picture is `HEAD -> None`. The length counter says zero because there are no boxes in the chain.
4. **Algorithm thinking.** Initialization should create a valid empty structure. Every later method can then ask one simple question: is `head` `None`? If yes, there is no first node to read, update, or delete.
5. **Pseudo code.**

```
create an empty LinkedList wrapper
set head to None because no first node exists
set n to 0 because the chain contains zero nodes
```

6. **Python implementation.**

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.n = 0
```

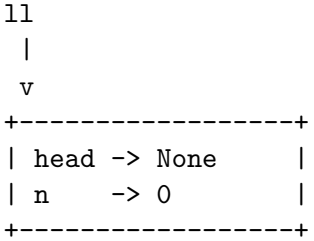
7. **Example program.**

```
ll = LinkedList()
print(ll.head)
print(ll.n)
```

8. **Console Output.**

```
None
0
```

9. **Explain output line-by-line.** The first output says there is no first node. The second says there are zero nodes.
10. **Memory Diagram Before.** No list wrapper exists.
11. **Pointer Changes.** Variable `ll` now references a `LinkedList`; `ll.head` references `None`.
12. **Memory Diagram After.**



13. Dry Run Table.

Step	Action	Result
1	create wrapper	object exists
2	assign head	None
3	assign n	0

14. Complexity. Time $O(1)$, space $O(1)$.

15. Common mistakes. Setting `head = Node(None)` creates a fake node; beginners should use `None`for empty.

16. Edge cases. Empty list is valid and should not crash methods.

17. Interview discussion. The list object is a handle; nodes are the chain.

18. Mini Exercise. Predict `ll.head is None` after construction.

`__len__`

Method focus: `__len__`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Users want `len(ll)` to tell how many nodes exist.
2. **Why we need this method.** Traversing every time is unnecessary if we maintain `n` during updates.
3. **Visual explanation.** The list wrapper keeps a small scoreboard named `n`. The nodes form the chain, while `n` records how many nodes the chain currently contains.
4. **Algorithm thinking.** Because every insert increments `n` and every delete decrements `n`, length does not need to walk through the chain. It simply reads the scoreboard.
5. **Pseudo code.**

```
when len(linked_list) is requested
read the stored counter n
return n without traversing nodes
```

6. **Python implementation.**

```
def __len__(self):
    return self.n
```

7. **Example program using existing `ll`.**

```
ll = LinkedList()
print(len(ll))
```

8. **Console Output.**

```
0
```

9. **Explain output line-by-line.** The empty list has `n = 0`, so `len(ll)` prints 0.
10. **Memory Diagram Before.**

```
HEAD -> None, n = 0
```

11. **Pointer Changes.** No pointer changes; this method only reads `n`.
12. **Memory Diagram After.**

```
HEAD -> None, n = 0
```

13. **Dry Run Table.**

Step	Read	Return
1	self.n	0

14. **Complexity.** Time $O(1)$, space $O(1)$.

15. **Common mistakes.** Forgetting to update `n` in insert/delete methods makes `len` lie.

16. **Edge cases.** Empty list returns 0; one-node list returns 1.

17. **Interview discussion.** Cached length trades small bookkeeping complexity for constant-time length.

18. **Mini Exercise.** If three insertions occur and one deletion occurs, what should `n` be?

is_empty

Method focus: `is_empty`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Many methods need to know if deletion or traversal is safe.
2. **Why we need this method.** Empty-list checks prevent attribute errors like `self.head.next` when `head` is `None`.
3. **Visual explanation.** The list is empty exactly when the `HEAD` arrow has no node to enter. If `head` is `None`, traversal would stop before visiting anything.
4. **Algorithm thinking.** Do not inspect `head.data` or `head.next`; those fields exist only if `head` points to a real node. The structural test is simply whether `head` points nowhere.
5. **Pseudo code.**

```
if head points to None
    return True
otherwise
    return False
```

6. **Python implementation.**

```
def is_empty(self):
    return self.head is None
```

7. **Example program using existing `ll`.**

```
print(ll.is_empty())
```

8. **Console Output.**

```
True
```

9. **Explain output line-by-line.** `ll.head` is `None`, so the method returns `True`.
10. **Memory Diagram Before.**

```
HEAD -> None
```

11. **Pointer Changes.** No pointer changes; only a comparison happens.
12. **Memory Diagram After.**

```
HEAD -> None
```

13. **Dry Run Table.**

Step	Check	Result
1	head is None	True

14. **Complexity.** $O(1)$ time and $O(1)$ space.

15. **Common mistakes.** Using `self.n == 0` is fine only if `n` is always maintained correctly.

16. **Edge cases.** Corrupted state `head != None` but `n = 0` indicates a bug.

17. **Interview discussion.** Prefer structural truth: `head is None`.

18. **Mini Exercise.** What should `is\_empty()` return after inserting one node?

display

Method focus: `display`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Learners need to see the list contents in order.
2. **Why we need this method.** Nodes are not stored in visible array positions; traversal reveals logical order.
3. **Visual explanation.** Imagine a finger named `current`. The finger starts where `head` points, reads the node's data, then slides along the `next` arrow. When the finger reaches `None`, the chain has ended.
4. **Algorithm thinking.** Never move `head` just to display the list. `head` is the permanent entrance to the chain; `current` is the temporary walker that may safely move from node to node.
5. **Pseudo code.**

```
current = head
while current is not None
    print current.data followed by an arrow
    move current to current.next
print None to show the end of the chain
```

6. **Python implementation.**

```
def display(self):
    current = self.head
    while current is not None:
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

7. **Example program using existing `ll`.**

```
ll.display()
```

8. **Console Output.**

```
None
```

9. **Explain output line-by-line.** The while loop never runs because `current` is `None`. The method prints the terminal marker `None`.
10. **Memory Diagram Before.**

```
HEAD -> None
```

11. **Pointer Changes.** `current` is a temporary reference; `head` does not move.
12. **Memory Diagram After.**

```
HEAD -> None
```

13. Dry Run Table.

Step	current	Printed
1	None	loop skipped
2	None	None

14. Complexity. $O(n)$ time because every node is visited; $O(1)$ extra space.

15. Common mistakes. Writing `self.head = self.head.next` during display destroys the list.

16. Edge cases. Empty list prints `None`; single-node list prints `value -> None`.

17. Interview discussion. Traversal is the basis for search, update, and many insertions/deletions.

18. Mini Exercise. Trace display for `10 -> 20 -> None`.

insert_at_beginning

Method focus: `insert_at_beginning`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Add a new first item quickly.
2. **Why we need this method.** Python list front insertion shifts many elements; linked list front insertion rewires two references.
3. **Visual explanation.** A new first node must stand in front of the old first node. Therefore, the new node's `next` arrow must point to the old head node before the list's `head` arrow moves to the new node.
4. **Algorithm thinking.** The old head is the doorway to the entire existing chain. If `head` moves too early, the old chain can be lost. The safe order is: create new node, connect new node to old chain, then move `head` to the new node.
5. **Pseudo code.**

```
create new_node containing data
point new_node.next to the current head
move head so it points to new_node
increase the stored length by 1
```

6. **Python implementation.**

```python
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    self.n += 1
```

7. **Example program using existing `ll`.**

```python
ll.insert_at_beginning(30)
ll.insert_at_beginning(20)
ll.insert_at_beginning(10)
ll.display()
print(len(ll))
```

8. **Console Output.**

```
10 -> 20 -> 30 -> None
3
```

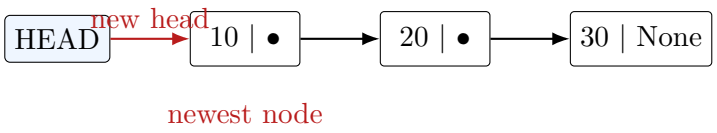
9. **Explain output line-by-line.** Insert 30 makes 30 first. Insert 20 points 20 to 30. Insert 10 points 10 to 20. Display follows head through 10, 20, 30, then `None`. Length is 3.
10. **Memory Diagram Before.**

```
HEAD -> None
```

11. **Pointer Changes.** For each insertion: create new node; set its `next` to old head; move `head` to new node; increment `n`.

12. **Memory Diagram After.**

```
HEAD
 |
 v
+------+------+
| 10   |  *---+---> +------+------+
+------+------+     | 20   |  *---+---> +------+------+
                    +------+------+     | 30   | None |
                                        +------+------+
```



13. **Dry Run Table.**

Operation	Old Head	New Node.next	New Head
insert 30	None	None	30
insert 20	30	30	20
insert 10	20	20	10

14. **Complexity.** $O(1)$ time and $O(1)$ extra space.

15. **Common mistakes.** Doing `self.head = new\_node` before `new\_node.next = self.head` makes the new node point to itself or loses the old list.

16. **Edge cases.** Works for empty, one-node, and many-node lists with the same code.

17. **Interview discussion.** This is the cleanest example of why linked lists can insert in $O(1)$ when location is known.

18. **Mini Exercise.** Starting with `A -> B`, insert `X` at beginning and draw pointer changes.

insert_at_end

Method focus: `insert_at_end`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Append a new item after the current last node.
2. **Why we need this method.** Building lists in natural order often needs append behavior.
3. **Visual explanation.** The last node is recognized by its `next` arrow pointing to `None`. Appending means replacing that final `None`arrow with an arrow to the new node.
4. **Algorithm thinking.** If the list is empty, there is no tail node to rewire, so `head` must point to the new node. Otherwise, traversal stops one node before `None`because that node is the one whose `next` reference must change.
5. **Pseudo code.**

```
create new_node containing data
if head is None
    head = new_node
    increase length
    stop
current = head
while current.next is not None
    current = current.next
set current.next to new_node
increase length
```

6. **Python implementation.**

```
def insert_at_end(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        self.n += 1
        return
    current = self.head
    while current.next is not None:
        current = current.next
    current.next = new_node
    self.n += 1
```

7. **Example program using existing `ll`.**

```
ll.insert_at_end(40)
ll.insert_at_end(50)
ll.display()
```

8. **Console Output.**

```
10 -> 20 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** The previous list was 10, 20, 30. Appending 40 changes 30.next from `None`to 40. Appending 50 changes 40.next from `None`to 50.

10. Memory Diagram Before.

```
HEAD -> 10 -> 20 -> 30 -> None
```

11. Pointer Changes. `current` walks without changing links. Only the last node's `next` changes.

12. Memory Diagram After.

```
HEAD -> 10 -> 20 -> 30 -> 40 -> 50 -> None
```

13. Dry Run Table.

Append	Traversal Stops At	Changed Reference	Result
40	30	30.next = 40	10,20,30,40
50	40	40.next = 50	10,20,30,40,50

14. Complexity. $O(n)$ time without a tail pointer; $O(1)$ space.

15. Common mistakes. Looping while `current is not None` overshoots and loses the last node reference.

16. Edge cases. Empty list sets head; non-empty list rewires old last node.

17. Interview discussion. Mention that adding a `tail` reference makes append $O(1)$ but requires more bookkeeping.

18. Mini Exercise. Why does the loop condition use `current.next is not None`?

insert_after

Method focus: `insert_after`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Insert a value immediately after a known value.
2. **Why we need this method.** Many edits are relative: add 25 after 20.
3. **Visual explanation.** Suppose `current` is the node 20 and 20 currently points to 30. To insert 25 after 20, 25 must first point to 30; then 20 can safely point to 25.
4. **Algorithm thinking.** This operation changes two arrows. The first arrow preserves the remainder of the list; the second arrow inserts the new node into the chain. Reversing this order risks disconnecting the nodes after `current`.
5. **Pseudo code.**

```
current = head
while current is not None
    if current.data equals target
        create new_node containing data
        point new_node.next to current.next
        point current.next to new_node
        increase length
        return True
    current = current.next
return False because target was not found
```

6. **Python implementation.**

```
def insert_after(self, target, data):
    current = self.head
    while current is not None:
        if current.data == target:
            new_node = Node(data)
            new_node.next = current.next
            current.next = new_node
            self.n += 1
            return True
        current = current.next
    return False
```

7. **Example program using existing `ll`.**

```
print(ll.insert_after(20, 25))
ll.display()
```

8. **Console Output.**

```
True
10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** `True` means target 20 was found. Display shows 25 immediately after 20 because 20.next now points to 25, and 25.next points to old 30.

10. Memory Diagram Before.

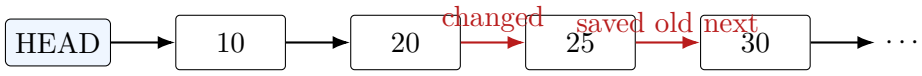
```
HEAD -> 10 -> 20 -> 30 -> 40 -> 50 -> None
```

11. Pointer Changes.

```
new(25).next = 30
20.next      = 25
```

12. Memory Diagram After.

```
HEAD -> 10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
```



13. Dry Run Table.

current	Compare	Action	Stop?
10	10==20 false	move	no
20	20==20 true	insert 25	yes

14. Complexity. $O(n)$ search time, $O(1)$ rewiring time.

15. Common mistakes. Setting `current.next = new\_node` first disconnects the old remainder unless saved.

16. Edge cases. Target absent returns `False`; inserting after last works because new.next becomes `None`.

17. Interview discussion. State the two-link order clearly: new-to-remainder, previous-to-new.

18. Mini Exercise. Insert 45 after 40 and draw the two assignments.

insert_before

Method focus: `insert_before`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Insert a value immediately before a known value.
2. **Why we need this method.** Singly lists cannot move backward, so we must remember the previous node while traversing.
3. **Visual explanation.** To insert before a node, you need the node itself and the node immediately before it. For `10 -> 20`, inserting 15 before 20 means changing 10's arrow to 15 and pointing 15's arrow to 20.
4. **Algorithm thinking.** A singly linked list cannot move backward, so traversal must carry two references: `previous` and `current`. If the target is already at `head`, there is no previous node, so the operation becomes insert-at-beginning.
5. **Pseudo code.**

```
if list is empty
    return False
if head.data equals target
    insert at beginning
    return True
previous = head
current = head.next
while current is not None
    if current.data equals target
        create new_node
        previous.next = new_node
        new_node.next = current
        increase length
        return True
    move previous and current one step forward
return False
```

6. **Python implementation.**

```
def insert_before(self, target, data):
    if self.head is None:
        return False
    if self.head.data == target:
        self.insert_at_beginning(data)
        return True
    previous = self.head
    current = self.head.next
    while current is not None:
        if current.data == target:
            new_node = Node(data)
            previous.next = new_node
            new_node.next = current
            self.n += 1
            return True
        previous = current
```

```
            current = current.next
        return False
```

7. **Example program using existing `ll`.**

```
print(ll.insert_before(20, 15))
ll.display()
```

8. **Console Output.**

```
True
10 -> 15 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** `True` means 20 was found. Display shows 15 before 20 because 10.next changed to 15, and 15.next points to 20.

10. **Memory Diagram Before.**

```
HEAD -> 10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
```

11. **Pointer Changes.**

```
previous = 10, current = 20
10.next = 15
15.next = 20
```

12. **Memory Diagram After.**

```
HEAD -> 10 -> 15 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
```

13. **Dry Run Table.**

previous	current	Compare	Action
10	20	true	insert between them

14. **Complexity.** $O(n)$ time, $O(1)$ space.

15. **Common mistakes.** Not handling target at head; then previous does not exist.

16. **Edge cases.** Empty list returns `False`; target at head delegates to beginning insertion.

17. **Interview discussion.** This method proves why singly lists often require `previous` and `current` pointers.

18. **Mini Exercise.** Insert 5 before 10. Which method should be reused?

insert_at_position

Method focus: `insert_at_position`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Insert at a numeric position like index 3.
2. **Why we need this method.** Sometimes the user knows position, not neighboring value.
3. **Visual explanation.** A position is a gap decision. To place a node at index `p`, the node that was at `p` must slide logically after the new node, so we stop at index `p-1` and rewire after that node.
4. **Algorithm thinking.** Validate before walking so traversal never goes beyond the list. Position 0 changes `head`; all other valid positions change the `next` field of the node just before the insertion point.
5. **Pseudo code.**

```
if position is less than 0 or greater than length
    raise an index error
if position is 0
    insert at beginning
    stop
previous = head
repeat position - 1 times
    previous = previous.next
create new_node
new_node.next = previous.next
previous.next = new_node
increase length
```

6. **Python implementation.**

```
def insert_at_position(self, position, data):
    if position < 0 or position > self.n:
        raise IndexError("position out of range")
    if position == 0:
        self.insert_at_beginning(data)
        return
    previous = self.head
    for _ in range(position - 1):
        previous = previous.next
    new_node = Node(data)
    new_node.next = previous.next
    previous.next = new_node
    self.n += 1
```

7. **Example program using existing `ll`.**

```
ll.insert_at_position(3, 18)
ll.display()
```

8. **Console Output.**

```
10 -> 15 -> 20 -> 18 -> 25 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** Position 3 means after positions 0, 1, and 2. The node at position 2 is 20, so 18 is inserted after 20 and before old 25.

10. **Memory Diagram Before.**

```
pos:   0    1    2    3    4    5    6
HEAD ->10 ->15 ->20 ->25 ->30 ->40 ->50 -> None
```

11. **Pointer Changes.**

```
previous = node(20)
new(18).next = 25
20.next = 18
```

12. **Memory Diagram After.**

```
pos:   0    1    2    3    4    5    6    7
HEAD ->10 ->15 ->20 ->18 ->25 ->30 ->40 ->50 -> None
```

13. **Dry Run Table.**

Loop Count	previous points to	Reason
start	10	position 0
1	15	move once
2	20	stop at p-1

14. **Complexity.** $O(n)$ time, $O(1)$ space.

15. **Common mistakes.** Walking to position `p` instead of `p-1`; allowing `position > n`.

16. **Edge cases.** Position 0, position `n`, negative position, empty list position 0.

17. **Interview discussion.** Always clarify whether positions are zero-based.

18. **Mini Exercise.** Insert 99 at position 0 and predict output.

search

Method focus: `search`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Find whether a value exists and where it appears.
2. **Why we need this method.** Linked lists do not support direct indexing; we must compare node by node.
3. **Visual explanation.** Searching is like checking each train car in order. `current` points to the car being inspected, while `position` counts how many arrows we followed from `head`.
4. **Algorithm thinking.** The first matching node determines the answer. If `current` becomes `None`, every node has been checked and the value is absent.
5. **Pseudo code.**

```
current = head
position = 0
while current is not None
    if current.data equals value
        return position
    current = current.next
    position = position + 1
return -1
```

6. **Python implementation.**

```
def search(self, value):
    current = self.head
    position = 0
    while current is not None:
        if current.data == value:
            return position
        current = current.next
        position += 1
    return -1
```

7. **Example program using existing `ll`.**

```
position = ll.search(30)
print("Found at position", position)
print(ll.search(999))
```

8. **Console Output.**

```
Found at position 5
-1
```

9. **Explain output line-by-line.** The value 30 is found after visiting positions 0 through 5. The value 999 is not found, so traversal stops at `None`and returns -1.
10. **Memory Diagram Before.**

```
HEAD -> 10 -> 15 -> 20 -> 18 -> 25 -> 30 -> 40 -> 50 -> None
```

11. **Pointer Changes.** No structural links change. Only temporary `current` moves.

12. **Memory Diagram After.**

```
HEAD -> 10 -> 15 -> 20 -> 18 -> 25 -> 30 -> 40 -> 50 -> None
```

13. **Dry Run Table.**

Position	current.data	Compare to 30
0	10	no
1	15	no
2	20	no
3	18	no
4	25	no
5	30	yes, return 5

14. **Complexity.** Best $O(1)$ if head matches; worst $O(n)$.

15. **Common mistakes.** Forgetting `position += 1`; checking `current.data` after `current` becomes `None`.

16. **Edge cases.** Empty list returns -1; duplicates return first match.

17. **Interview discussion.** Search is linear because nodes only know the next node.

18. **Mini Exercise.** Trace `search(18)` and list comparisons.

update

Method focus: `update`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Replace an existing value without changing list structure.
2. **Why we need this method.** Sometimes data changes but node order should remain identical.
3. **Visual explanation.** Updating changes the label inside a box, not the arrows between boxes. The chain order remains exactly the same.
4. **Algorithm thinking.** Traverse exactly like search. When the old value is found, mutate only `current.data`; do not create a new node and do not touch `current.next`.
5. **Pseudo code.**

```
current = head
while current is not None
    if current.data equals old_value
        current.data = new_value
        return True
    current = current.next
return False
```

6. **Python implementation.**

```
def update(self, old_value, new_value):
    current = self.head
    while current is not None:
        if current.data == old_value:
            current.data = new_value
            return True
        current = current.next
    return False
```

7. **Example program using existing `ll`.**

```
print(ll.update(18, 22))
ll.display()
```

8. **Console Output.**

```
True
10 -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** `True` confirms 18 was found. The list now shows 22 in the same position where 18 used to be.
10. **Memory Diagram Before.**

```
HEAD -> 10 -> 15 -> 20 -> 18 -> 25 -> 30 -> 40 -> 50 -> None
```

11. **Pointer Changes.** No `next` pointer changes. Only `node.data` changes from 18 to 22.

12. Memory Diagram After.

```
HEAD -> 10 -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None
```

13. Dry Run Table.

current.data	Match 18?	Action
10	no	move
15	no	move
20	no	move
18	yes	change to 22

14. Complexity. $O(n)$ time, $O(1)$ space.

15. Common mistakes. Creating a new node unnecessarily; changing links when only data must change.

16. Edge cases. Empty list returns `False`; duplicates update first match only.

17. Interview discussion. Make clear whether all matches or first match should be updated.

18. Mini Exercise. Modify the method to update all matching values.

delete_first

Method focus: `delete_first`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Remove the first item, like dequeueing from the front.
2. **Why we need this method.** Front deletion is where linked lists shine: move `head` once.
3. **Visual explanation.** The first node is removed by moving the `HEAD` arrow forward one step. The old first node is no longer reachable from the list.
4. **Algorithm thinking.** Read the deleted value before moving `head`. Then redirect `head` to the second node. If there is no second node, `head` naturally becomes `None`.
5. **Pseudo code.**

```
if head is None
    return None
store head.data as deleted_value
move head to head.next
decrease length
return deleted_value
```

6. **Python implementation.**

```
def delete_first(self):
    if self.head is None:
        return None
    deleted_value = self.head.data
    self.head = self.head.next
    self.n -= 1
    return deleted_value
```

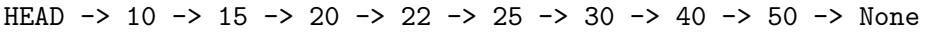
7. **Example program using existing `ll`.**

```
print(ll.delete_first())
ll.display()
```

8. **Console Output.**

```
10
15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None
```

9. **Explain output line-by-line.** The deleted value is 10. Display starts at 15 because `head` changed from node 10 to node 15.
10. **Memory Diagram Before.**



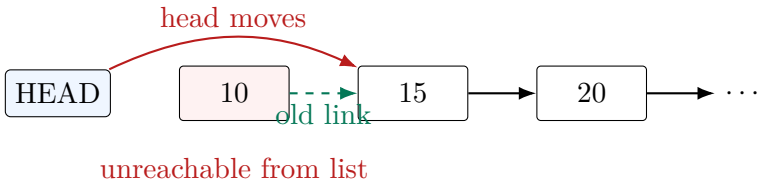
11. **Pointer Changes.**

```
old head = 10
new head = old_head.next = 15
node 10 becomes unreachable from the list
```

Python's garbage collector can reclaim node 10 when no references to it remain.

12. Memory Diagram After.

```
HEAD -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None
10 is no longer reachable from HEAD
```



13. Dry Run Table.

Step	Reference	Value
1	head.data	10
2	head.next	15
3	head	moved to 15

14. Complexity. $O(1)$ time, $O(1)$ space.

15. Common mistakes. Accessing `self.head.data` before checking empty list.

16. Edge cases. Empty returns `None`; one-node list moves head to `None`.

17. Interview discussion. This is a constant-time deletion because no traversal is needed.

18. Mini Exercise. What is head after deleting first from `A -> None`?

delete_last

Method focus: `delete_last`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Remove the tail node.
2. **Why we need this method.** Singly lists cannot jump backward from the last node to its previous node.
3. **Visual explanation.** The last node cannot remove itself from the chain because no node points backward. We must find the node before the last node and change that node's `next` arrow to `None`.
4. **Algorithm thinking.** Empty lists have nothing to delete. Single-node lists delete by moving `head` to `None`. Longer lists require two moving references: `previous` trails behind `current` until `current` reaches the last node.
5. **Pseudo code.**

```
if head is None
    return None
if head.next is None
    store head.data
    head = None
    decrease length
    return stored value
previous = head
current = head.next
while current.next is not None
    previous = current
    current = current.next
store current.data
previous.next = None
decrease length
return stored value
```

6. **Python implementation.**

```
def delete_last(self):
    if self.head is None:
        return None
    if self.head.next is None:
        deleted_value = self.head.data
        self.head = None
        self.n -= 1
        return deleted_value
    previous = self.head
    current = self.head.next
    while current.next is not None:
        previous = current
        current = current.next
    deleted_value = current.data
    previous.next = None
    self.n -= 1
    return deleted_value
```

7. **Example program using existing `ll`.**

```
print(ll.delete_last())
ll.display()
```

8. **Console Output.**

```
50
15 -> 20 -> 22 -> 25 -> 30 -> 40 -> None
```

9. **Explain output line-by-line.** The deleted tail value is 50. The node 40 becomes the new last node because 40.next is set to `None`.

10. **Memory Diagram Before.**

```
HEAD -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None
```

11. **Pointer Changes.**

```
previous = 40, current = 50
40.next = None
50 becomes unreachable from HEAD
```

Python's garbage collector can reclaim node 50 if no external reference exists.

12. **Memory Diagram After.**

```
HEAD -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> None
```

13. **Dry Run Table.**

previous	current	current.next?
15	20	not None
20	22	not None
22	25	not None
25	30	not None
30	40	not None
40	50	None, stop

14. **Complexity.** $O(n)$ time, $O(1)$ space.

15. **Common mistakes.** Setting `current = None` only changes the local variable, not the list link.

16. **Edge cases.** Empty list, one-node list, many-node list.

17. **Interview discussion.** Without a previous pointer or tail predecessor, deleting tail is linear.

18. **Mini Exercise.** Why do we need `previous.next = None` instead of `current = None`?

`delete_by_value`

Method focus: `delete_by_value`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Remove the first node containing a specific value.
2. **Why we need this method.** Users often know the value to remove, not its position.
3. **Visual explanation.** Deletion by value bypasses the matching node. In `20 -> 22 -> 25`, the node 20 should stop pointing to 22 and start pointing directly to 25.
4. **Algorithm thinking.** The node before the match owns the arrow that must change. Therefore, after handling the head case, traversal keeps both `previous` and `current` so `previous.next` can skip `current`.
5. **Pseudo code.**

```
if head is None
    return False
if head.data equals value
    delete first node
    return True
previous = head
current = head.next
while current is not None
    if current.data equals value
        previous.next = current.next
        decrease length
        return True
    previous = current
    current = current.next
return False
```

6. **Python implementation.**

```
def delete_by_value(self, value):
    if self.head is None:
        return False
    if self.head.data == value:
        self.delete_first()
        return True
    previous = self.head
    current = self.head.next
    while current is not None:
        if current.data == value:
            previous.next = current.next
            self.n -= 1
            return True
        previous = current
        current = current.next
    return False
```

7. **Example program using existing `ll`.**

```
print(ll.delete_by_value(22))
ll.display()
```

8. **Console Output.**

```
True
15 -> 20 -> 25 -> 30 -> 40 -> None
```

9. **Explain output line-by-line.** `True` confirms 22 was found and removed. The list jumps from 20 directly to 25.

10. **Memory Diagram Before.**

```
HEAD -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> None
```

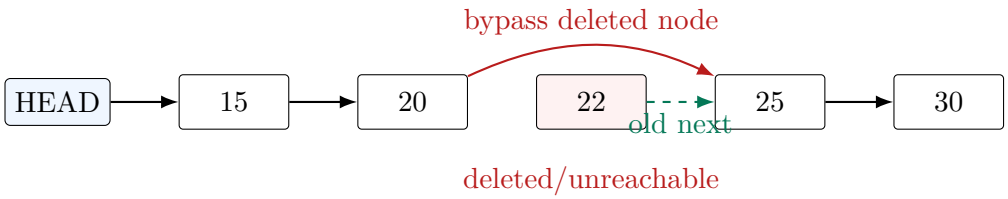
11. **Pointer Changes.**

```
previous = 20, current = 22
20.next = 22.next = 25
22 becomes unreachable from HEAD
```

Python's garbage collector reclaims node 22 later if no references remain.

12. **Memory Diagram After.**

```
HEAD -> 15 -> 20 -> 25 -> 30 -> 40 -> None
```



13. **Dry Run Table.**

previous	current	Match 22?	Action
15	20	no	move
20	22	yes	bypass 22

14. **Complexity.** $O(n)$ time, $O(1)$ space.

15. **Common mistakes.** Forgetting to decrement `n`; not handling deletion at head.

16. **Edge cases.** Empty list, missing value, duplicate values, deleting tail.

17. **Interview discussion.** Clarify whether to delete first occurrence or all occurrences.

18. **Mini Exercise.** Modify the algorithm to delete all 25 values.

delete_by_position

Method focus: `delete_by_position`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Remove the node at a zero-based position.
2. **Why we need this method.** Position-based APIs are familiar from arrays and Python lists.
3. **Visual explanation.** Deleting by position removes the node at a counted location. For any position after 0, the node at `position - 1` must point around the deleted node.
4. **Algorithm thinking.** Invalid positions should fail before pointer movement begins. Position 0 changes `head`; all other positions change `previous.next`.
5. **Pseudo code.**

```
if position is less than 0 or position is at least length
    raise an index error
if position is 0
    delete first node and return its value
previous = head
repeat position - 1 times
    previous = previous.next
deleted_node = previous.next
previous.next = deleted_node.next
decrease length
return deleted_node.data
```

6. **Python implementation.**

```
def delete_by_position(self, position):
    if position < 0 or position >= self.n:
        raise IndexError("position out of range")
    if position == 0:
        return self.delete_first()
    previous = self.head
    for _ in range(position - 1):
        previous = previous.next
    deleted_node = previous.next
    previous.next = deleted_node.next
    self.n -= 1
    return deleted_node.data
```

7. **Example program using existing `ll`.**

```
print(ll.delete_by_position(2))
ll.display()
```

8. **Console Output.**

```
25
15 -> 20 -> 30 -> 40 -> None
```

9. **Explain output line-by-line.** Position 2 contains 25, so 25 is returned. The displayed list now connects 20 directly to 30.

10. **Memory Diagram Before.**

```
pos:   0    1    2    3    4
HEAD ->15 ->20 ->25 ->30 ->40 -> None
```

11. **Pointer Changes.**

```
previous = 20
deleted_node = 25
20.next = 25.next = 30
25 becomes unreachable from HEAD
```

12. **Memory Diagram After.**

```
pos:   0    1    2    3
HEAD ->15 ->20 ->30 ->40 -> None
```

13. **Dry Run Table.**

Loop Count	previous	Purpose
start	15	position 0
1	20	stop at p-1

14. **Complexity.** $O(n)$ time, $O(1)$ space.

15. **Common mistakes.** Allowing `position == n`; valid last index is `n-1`.

16. **Edge cases.** Delete first, delete last, invalid negative index, invalid too-large index.

17. **Interview discussion.** Carefully define indexing and exception behavior.

18. **Mini Exercise.** Delete position 0 from the current list and predict returned value.

`reverse`

Method focus: `reverse`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Reverse the order of the list in place.
2. **Why reversing is difficult.** Each node only points forward. If you redirect `current.next` backward without first saving the original next node, the rest of the list is lost.
3. **Need.** We need three temporary references: `previous`, `current`, and `next_node`.
4. **Visual explanation.** Reversal splits the list into two zones. `previous` points to the already reversed prefix, `current` points to the node whose arrow is about to flip, and `next_node` temporarily protects the unreversed suffix from being lost.
5. **Algorithm thinking.** Each iteration must happen in a safe order: first save the original next node, then flip `current.next` backward, then move `previous` and `current` one step forward. This invariant means `previous` is always the head of the reversed part.
6. **Visualization before code.**

```
Before loop:
previous -> None
current  -> 15 -> 20 -> 30 -> 40 -> None

After one iteration:
previous -> 15 -> None
current  -> 20 -> 30 -> 40 -> None
```

7. **Logic.** Save the next node before changing any arrow; flip exactly one arrow; advance both temporary references.
8. **Dry Run before code.**

Iteration	next_node	Changed Link	previous after	current after
1	20	15.next=None	15	20
2	30	20.next=15	20	30
3	40	30.next=20	30	40
4	None	40.next=30	40	None

9. **Pseudo code.**

```
previous = None
current = head
while current is not None
    next_node = current.next # save the unreversed remainder
    current.next = previous # flip the current arrow backward
    previous = current # grow the reversed prefix
    current = next_node # move into the unreversed suffix
head = previous # old tail is now the new head
```

10. Python implementation.

```
def reverse(self):
    previous = None
    current = self.head
    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node
    self.head = previous
```

11. Example program using existing `ll`.

```
ll.reverse()
ll.display()
```

12. Console Output.

```
40 -> 30 -> 20 -> 15 -> None
```

13. Explain output line-by-line. The original list was 15, 20, 30, 40. Each arrow is reversed. The old tail 40 becomes the new head, so traversal prints 40, 30, 20, 15.

14. Memory Diagram Before.

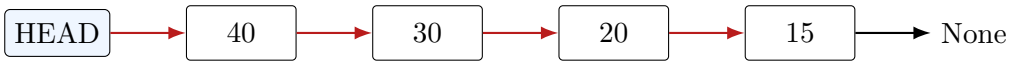
```
HEAD -> 15 -> 20 -> 30 -> 40 -> None
```

15. Pointer Movement Visualization.

```
Iteration 1: None <- 15    20 -> 30 -> 40 -> None
Iteration 2: None <- 15 <- 20    30 -> 40 -> None
Iteration 3: None <- 15 <- 20 <- 30    40 -> None
Iteration 4: None <- 15 <- 20 <- 30 <- 40
HEAD moves to previous, which is 40
```

16. Memory Diagram After.

```
HEAD -> 40 -> 30 -> 20 -> 15 -> None
```



all arrows now point in the reversed direction

17. Dry Run Table.

current	next saved	current.next becomes	reversed prefix
15	20	None	15
20	30	15	20 -> 15
30	40	20	30 -> 20 -> 15
40	None	30	40 -> 30 -> 20 -> 15

18. **Complexity.** $O(n)$ time, $O(1)$ extra space.

19. **Common mistakes.** Forgetting `next_node = current.next` before rewiring; forgetting `self.head = previous` at the end.

20. **Edge cases.** Empty list remains empty; one-node list remains unchanged; two-node list swaps links.

21. **Interview discussion.** Explain the invariant: `previous` always points to the fully reversed prefix.

22. **Mini Exercise.** Reverse `A -> B -> C -> None` by drawing each iteration.

clear

Method focus: `clear`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Remove all nodes from the list.
2. **Why we need this method.** Users need a reset operation.
3. **Visual explanation.** Clearing cuts the list's entrance arrow. Once `head` points to `None`, the list wrapper can no longer reach any old node.
4. **Algorithm thinking.** Python manages memory automatically, so we do not need to manually visit and free each node. We only detach the chain and reset the length counter.
5. **Pseudo code.**

```
head = None
n = 0
old nodes become unreachable from this list
```

6. **Python implementation.**

```
def clear(self):
    self.head = None
    self.n = 0
```

7. **Example program using existing `ll`.**

```
ll.clear()
ll.display()
print(len(ll))
```

8. **Console Output.**

```
None
0
```

9. **Explain output line-by-line.** Display prints `None`because head is now `None`. Length prints 0 because `n` was reset.
10. **Memory Diagram Before.**

```
HEAD -> 40 -> 30 -> 20 -> 15 -> None
```

11. **Pointer Changes.**

```
HEAD = None
All old nodes become unreachable from ll unless external references exist.
```

Python's garbage collector can reclaim unreachable nodes.

12. **Memory Diagram After.**

```
HEAD -> None, n = 0
```

13. Dry Run Table.

Step	Assignment	Effect
1	head=None	chain detached
2	n=0	length reset

14. Complexity. $O(1)$ direct work; garbage collection cost is managed by Python runtime.

15. Common mistakes. Traversing to delete each node manually is unnecessary in Python for this beginner implementation.

16. Edge cases. Clearing an already empty list is safe.

17. Interview discussion. In languages with manual memory management, each node may need explicit freeing.

18. Mini Exercise. What happens if another variable still points to an old node?

to_list

Method focus: `to_list`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Convert the linked list into a normal Python list.
2. **Why we need this method.** Python lists are convenient for testing, printing, serialization, and comparison.
3. **Visual explanation.** `to_list` copies the values out of the node chain into a normal Python list. The linked list remains unchanged; only a new array-style container is produced.
4. **Algorithm thinking.** Traverse with `current`. At each node, append `current.data` to `result`. Never append `current` itself unless the caller explicitly wants node objects.
5. **Pseudo code.**

```
result = empty Python list
current = head
while current is not None
    append current.data to result
    current = current.next
return result
```

6. **Python implementation.**

```
def to_list(self):
    result = []
    current = self.head
    while current is not None:
        result.append(current.data)
        current = current.next
    return result
```

7. **Example program reusing `ll` after clear.**

```
ll.insert_at_end(5)
ll.insert_at_end(6)
ll.insert_at_end(7)
print(ll.to_list())
```

8. **Console Output.**

```
[5, 6, 7]
```

9. **Explain output line-by-line.** The method visits 5, then 6, then 7, appending each value into a normal Python list.
10. **Memory Diagram Before.**

```
HEAD -> 5 -> 6 -> 7 -> None
result = []
```

11. **Pointer Changes.** No list links change. Temporary `current` moves; `result` grows.

12. **Memory Diagram After.**

```
HEAD -> 5 -> 6 -> 7 -> None
result = [5, 6, 7]
```

13. **Dry Run Table.**

current.data	Append	result
5	yes	[5]
6	yes	[5,6]
7	yes	[5,6,7]

14. **Complexity.** $O(n)$ time and $O(n)$ output space.

15. **Common mistakes.** Returning nodes instead of data values when the API promises a Python list of values.

16. **Edge cases.** Empty linked list returns `[]`.

17. **Interview discussion.** Useful for testing because expected arrays are easy to compare.

18. **Mini Exercise.** What should `LinkedList().to\_list()` return?

from_list

Method focus: `from_list`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Build a linked list from an existing Python list.
2. **Why we need this method.** It makes testing and setup faster.
3. **Visual explanation.** `from_list` takes values from a normal Python list and creates one linked-list node per value, preserving left-to-right order.
4. **Algorithm thinking.** This is a construction helper, so it should create a fresh list rather than unexpectedly modifying an existing one. Appending each value keeps the final linked-list order identical to the input order.
5. **Pseudo code.**

```
create a new empty linked list
for each value in the input Python list
    insert the value at the end of the linked list
return the completed linked list
```

6. **Python implementation.**

```
@classmethod
def from_list(cls, values):
    linked_list = cls()
    for value in values:
        linked_list.insert_at_end(value)
    return linked_list
```

7. **Example program using a fresh object because this is a constructor helper.**

```
ll = LinkedList.from_list([100, 200, 300])
ll.display()
print(len(ll))
```

8. **Console Output.**

```
100 -> 200 -> 300 -> None
3
```

9. **Explain output line-by-line.** Values are appended in the same order as the Python list. Length is 3 because three nodes were created.
10. **Memory Diagram Before.**

```
values = [100, 200, 300]
new linked list: HEAD -> None
```

11. **Pointer Changes.** First value becomes head. Each later value is attached to the old tail's `next`.
12. **Memory Diagram After.**

```
HEAD -> 100 -> 200 -> 300 -> None
```

13. Dry Run Table.

Value	Insertion	List
100	empty head	100
200	append	100 -> 200
300	append	100 -> 200 -> 300

14. Complexity. With this no-tail implementation, repeated append is $O(n^2)$ for long input. With a tail pointer it can be $O(n)$.

15. Common mistakes. Making it an instance method that mutates an existing list when the API intends construction.

16. Edge cases. Empty input returns an empty linked list.

17. Interview discussion. Mention tail optimization if asked about performance.

18. Mini Exercise. Build from `[]` and predict `display()`.

`__iter__`

Method focus: `__iter__`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Users want `for value in ll` to work naturally.
2. **Why we need this method.** Python iteration protocols make custom containers feel built-in.
3. **Visual explanation.** Iteration turns the linked list into a sequence Python can loop over. Each `yield` hands one value to the caller, then traversal resumes from the same node on the next loop step.
4. **Algorithm thinking.** The iterator should expose values, not internal node objects. It uses the same safe temporary pointer pattern as display and search.
5. **Pseudo code.**

```
current = head
while current is not None
    yield current.data to the caller
    current = current.next
```

6. **Python implementation.**

```
def __iter__(self):
    current = self.head
    while current is not None:
        yield current.data
        current = current.next
```

7. **Example program using existing `ll`.**

```
for value in ll:
    print(value)
```

8. **Console Output.**

```
100
200
300
```

9. **Explain output line-by-line.** The loop receives 100, then 200, then 300 from successive `yield` statements.
10. **Memory Diagram Before.**

```
HEAD -> 100 -> 200 -> 300 -> None
```

11. **Pointer Changes.** No structural changes. The generator's local `current` moves forward.
12. **Memory Diagram After.**

```
HEAD -> 100 -> 200 -> 300 -> None
```

13. Dry Run Table.

current	yielded	next current
100	100	200
200	200	300
300	300	None

14. Complexity. Full iteration is $O(n)$; each yielded step is $O(1)$.

15. Common mistakes. Yielding nodes when users expect values; accidentally modifying `head`.

16. Edge cases. Empty list yields nothing.

17. Interview discussion. Iterators separate traversal behavior from storage details.

18. Mini Exercise. Use `list(ll)` and predict the result.

`__str__`

Method focus: `__str__`. Read this operation as a pointer story: identify the old link, preserve any node that must not be lost, then redirect exactly the reference that changes.

1. **Real-world problem.** Users want `print(ll)` to show readable structure.
2. **Why we need this method.** Debugging pointer structures is easier with a stable string representation.
3. **Visual explanation.** The string form is a readable picture of traversal order. Each node value becomes text, arrows show `next` links, and `None`marks the final missing successor.
4. **Algorithm thinking.** Reuse `__iter__` so the class has one canonical way to walk through values. String formatting should not mutate pointers or change length.
5. **Pseudo code.**

```
convert each iterated value to a string
join the strings using " -> "
append " -> None" to show the list terminator
return the completed string
```

6. **Python implementation.**

```
def __str__(self):
    return " -> ".join(str(value) for value in self) + " -> None"
```

7. **Example program using existing `ll`.**

```
print(ll)
```

8. **Console Output.**

```
100 -> 200 -> 300 -> None
```

9. **Explain output line-by-line.** `print` calls `__str__`. Iteration provides 100, 200, and 300, which are joined with arrows.
10. **Memory Diagram Before.**

```
HEAD -> 100 -> 200 -> 300 -> None
```

11. **Pointer Changes.** No pointer changes. Only a string is constructed.
12. **Memory Diagram After.**

```
HEAD -> 100 -> 200 -> 300 -> None
```

13. **Dry Run Table.**

Iterated values	Joined text	Final suffix
100,200,300	100 -> 200 -> 300	-> None

14. **Complexity.** $O(n)$ time and $O(n)$ string space.

15. **Common mistakes.** Forgetting the empty-list case: this implementation returns `" -> None"` for empty. A polished version can return `"None"` when empty.

16. **Edge cases.** Empty list formatting should be decided intentionally.

17. **Interview discussion.** `__str__` is not algorithmically essential, but it improves observability.

18. **Mini Exercise.** Improve `__str__` so an empty list prints exactly `None`.

1.8 One Continuous Evolution of `ll`

Step	Operation	List After Operation
1	create	`None`
2	insert beginning 30, 20, 10	`10 -> 20 -> 30 -> None`
3	append 40, 50	`10 -> 20 -> 30 -> 40 -> 50 -> None`
4	insert 25 after 20	`10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None`
5	insert 15 before 20	`10 -> 15 -> 20 -> 25 -> 30 -> 40 -> 50 -> None`
6	insert 18 at position 3	`10 -> 15 -> 20 -> 18 -> 25 -> 30 -> 40 -> 50 -> None`
7	update 18 to 22	`10 -> 15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None`
8	delete first	`15 -> 20 -> 22 -> 25 -> 30 -> 40 -> 50 -> None`
9	delete last	`15 -> 20 -> 22 -> 25 -> 30 -> 40 -> None`
10	delete value 22	`15 -> 20 -> 25 -> 30 -> 40 -> None`
11	delete position 2	`15 -> 20 -> 30 -> 40 -> None`
12	reverse	`40 -> 30 -> 20 -> 15 -> None`
13	clear	`None`

1.9 Summary

A singly linked list stores a sequence as a chain of nodes. The only entry point is `head`. Every node has `data` and `next`. Most linked-list operations are pointer rewiring operations. The safest way to reason is to identify the node before the change, the node after the change, and the reference that must be redirected.

1.10 Cheat Sheet

- Empty list: `head is None`
- Last node: `current.next is None`
- Insert at front: `new.next=head; head=new`
- Insert after: `new.next=current.next; current.next=new`
- Insert before: keep `previous` and `current`
- Delete first: `head=head.next`
- Delete middle: `previous.next=current.next`

- Delete last: `previous.next=None`
- Reverse: save next, flip link, advance two pointers
- Never move `head` for traversal; use `current`

1.11 Interview Notes

Interview Perspective

- Insertion/deletion is $O(1)$ only after you already have the relevant node reference.
- Searching is $O(n)$ because there is no direct index access.
- A singly linked list cannot move backward unless you store a `previous` pointer during traversal.
- Reversal requires preserving the original next node before flipping the current link.
- Python references behave like safe pointers for linked-list reasoning.

1.12 Debugging Tips

Tip

When a linked list breaks, ask four questions:

1. Did I save the rest of the list before overwriting a `next` reference?
2. Did I update `head` when the first node changed?
3. Did I update `n` exactly once?
4. Did I stop traversal before `current` became `None`?

1.13 Common Mistakes

Common Mistake

- Using `self.head` as the traversal pointer and accidentally destroying the entry point.
- Forgetting empty-list and one-node-list cases.
- Rewiring in the wrong order during insertion.
- Setting a local variable to `None`instead of changing the previous node's `next` field.
- Forgetting that deleted nodes are garbage-collected only after no references remain.

1.14 Revision Questions

1. What does `head` store?

2. Why does display use `current` instead of changing `head`?
3. Why does insertion after a node set `new.next` before `current.next`?
4. Why does deleting a middle node require a `previous` pointer?
5. Why is linked-list search $O(n)$?
6. What are the three pointers used in reversal?
7. What exact reference changes during `delete_first`?
8. What exact reference changes during `delete_last`?

1.15 Practice Problems

1. Implement `count(value)` to count occurrences.
2. Implement `get(position)` to return a value at an index.
3. Implement `middle()` using slow and fast pointers.
4. Implement `remove\_all(value)`.
5. Implement `has\_cycle()` using Floyd's algorithm.
6. Implement `nth\_from\_end(k)`.

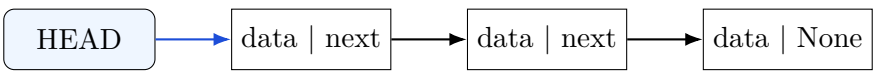
1.16 Expected Output Questions

1. Start empty. Run `insert\_at\_beginning(2)`, `insert\_at\_beginning(1)`, `insert\_at\_end(3)`. What does display print?
2. For `10 -> 20 -> 30 -> None`, what prints after `delete\_by\_value(20)`?
3. For `A -> B -> C -> None`, what prints after `reverse()`?
4. What should `search(99)` return when 99 is absent?

1.17 Pointer Tracing Questions

1. Draw the pointers changed by inserting X after B in `A -> B -> C -> None`.
2. Draw the pointers changed by deleting B from `A -> B -> C -> None`.
3. During reverse, what happens if you forget `next_node = current.next`?
4. In `delete_last`, why must the loop stop when `current.next is None`?

1.18 Final Mental Model



To master linked lists, stop asking "What code should I memorize?" and start asking "Which reference must change so the chain says the new story?"